

Test Logs And Defects

This file collates the full content of the main log, subagent logs, and confirmed defect files for the 2026-06-22 issue #233 / PR #234 exploratory review.

issue-233-test-log.md

2026-06-22 13:34 +01:00

- Prove live browser interaction with MCP before substantive testing so the session meets the explicit browser-access requirement and starts from a real deployed-environment baseline.

the actions you take - include urls, steps and data that you use, describe any UI interactions that you make

- Used Playwright MCP to open <https://eviltester.github.io/grid-table-editor/>.
- Confirmed the page title **Grid Table Editor Test Environment**.
- Captured the live page snapshot showing the root cards for `app.html`, `generator.html`, `combinatorial.html`, `webmcp.html`, `writer-schema.html`, Storybook, and `site/`.

the observations and results that you make

- Live browser access is confirmed for this fresh session via Playwright MCP.
 - The deployed root page is reachable and interactive, so substantive test-environment-only exploratory work can proceed.
 - The root page also exposes current build metadata including branch `233-test-env-consistency`, commit `ec1b8db43eb9`, and build timestamp `2026-06-22T12:16:58.547Z`.
-

2026-06-22 13:34 +01:00

- Re-read the current story, PR, and issue comments before broader testing so the fresh retest uses the latest expectations instead of blindly repeating older findings.

the actions you take - include urls, steps and data that you use, describe any UI interactions that you make

- Retrieved the live issue body for <https://github.com/eviltester/grid-table-editor/issues/233>.
- Retrieved the current PR metadata for <https://github.com/eviltester/grid-table-editor/pull/234>.

- Retrieved current issue and PR comments, including the issue comments that summarize which earlier AI findings were fixed, accepted, or still considered valid.
- Pulled the current changed-file inventory for PR #234.

the observations and results that you make

- The PR has moved beyond the earlier 28-file shape and now spans 49 changed files.
- The issue comments materially change the retest boundary:
 - the pre-JS Regex-link report was explicitly invalidated and the implementation was changed to generate correct URLs earlier
 - quick-start hosted links in docs were explicitly called out as acceptable
 - stale **abbreviated** examples were said to be amended
 - reversed-bounds false-success behavior was explicitly accepted as a real issue needing a fix
 - keyboard reachability of help-popup links was explicitly accepted as needing investigation
- This means the fresh retest must verify-after-fix rather than simply replaying yesterday's defect list.

2026-06-22 00:00 +01:00

- Session scaffold created so the fresh 2026-06-22 retest starts with the required append-only artifact structure before substantive testing begins.

the actions you take - include urls, steps and data that you use, describe any UI interactions that you make

- Created docs/testing/20260622/issue-233-001/ with defects/ and screenshots/.
- Wrote the session goal prompt into issue-233-session-goal-prompt.md.

the observations and results that you make

- The fresh session folder is ready for the required planning, delegation, logs, and defect packaging.

2026-06-22 13:35 +01:00

- I want to lock in the delegated test lanes before broad live testing begins so each required area has a clear owner, an append-only evidence file, and a non-overlapping scope.

the actions you take - include urls, steps and data that you use, describe any UI interactions that you make

- Created fresh subagent log files in docs/testing/20260622/issue-233-001/:
 - command-coverage-test-log.md
 - negative-validation-test-log.md
 - docs-consistency-test-log.md
 - ux-regression-test-log.md
 - responsive-accessibility-test-log.md
 - routing-consistency-test-log.md
- Added a written charter, owned write scope, and planned techniques/heuristics to each file.
- No application-under-test UI interaction was performed in this step; this was session setup in the artifact folder.

the observations and results that you make

- The required delegated coverage lanes are now physically present in the output folder and ready for live evidence capture.
- Substantive Loop 1 testing can now proceed with owned append-only logs instead of ad hoc notes.

2026-06-22 13:42 +01:00

- I want to verify the previously disputed Regex/help routing observations against the live runtime so Loop 1 starts by separating disproved older findings from still-open risks.

the actions you take - include urls, steps and data that you use, describe any UI interactions that you make

- Opened <https://eviltester.github.io/grid-table-editor/generator.html> in Playwright.
- Observed the default first schema row with **regex** selected and inspected the visible **Regex data help** link.
- Clicked **Regex data help** on the standalone generator and observed that it opened a docs tab at <https://eviltester.github.io/grid-table-editor/site/docs/test-data/regex>
- Opened the nested generator at <https://eviltester.github.io/grid-table-editor/site/generator.html>
- Observed the default first schema row on the nested generator and inspected the visible **Regex data help** link there.
- Opened the nested docs page <https://eviltester.github.io/grid-table-editor/site/docs/test-data/regex> and clicked the visible **App** navigation link, which opened <https://eviltester.github.io/grid-table-editor/app.html> in a new tab.
- Attempted to navigate via the root **Open site/** card once and hit a transient **ERR_CONNECTION_RESET**, then continued by directly opening the nested site URLs in fresh tabs.
- Opened <https://eviltester.github.io/grid-table-editor/writer-schema.html> and confirmed the shared schema editor surface loads, although the lower row-level controls were not yet fully exercised in this pass.

the observations and results that you make

- The previously disputed standalone `generator.html` Regex help issue does not reproduce in the current runtime. The visible help link and the clicked destination both point to the nested docs path under `/site/docs/test-data/regex-test-data/`.
- The nested `site/generator.html` surface also shows the `Regex data help` link targeting the nested `/site/docs/...` route.
- The nested docs page opens the nested `site/app.html` route as expected from its `App` navigation link.
- A one-off `ERR_CONNECTION_RESET` occurred when opening `site/` from the root card, but direct navigation to the nested site URLs succeeded immediately afterward, so this currently looks like an environment or network flake rather than a confirmed product defect.
- This means the older pre-JS Regex/help routing concern is not standing up as a current defect in Loop 1.

2026-06-22 13:46 +01:00

- I want to fold the first returned subagent evidence into Loop 1 so the report captures both the disproved old routing concern and the current state of the help/picker UX changes from PR #234.

the actions you take - include urls, steps and data that you use, describe any UI interactions that you make

- Reviewed the returned `ux-regression-test-log.md` from the delegated UX lane.
- Recorded the subagent's exercised path on `https://eviltester.github.io/grid-table-editor/genera` including:
 - switching the first row from `regex` to `faker`
 - opening the method picker and reviewing `helpers.arrayElement`
 - switching to the `internet` category
 - selecting `internet.httpMethod`
 - opening the params editor for `awd.domain.internet.httpMethod`
 - retesting multiple help/tippy triggers in sequence

the observations and results that you make

- The delegated UX lane found no regression in the exercised method-picker flow.
- The delegated UX lane found no regression in the exercised params-editor launch path for `internet.httpMethod`.
- The delegated UX lane provides positive evidence that the “only one tippy open at a time” behavior is working in the exercised generator help flows, with later help popups replacing earlier ones instead of stacking.

- The UX lane also surfaced one uncharacterized landing-page console error as a follow-up risk, but not a confirmed product defect in this loop.

2026-06-22 13:46 +01:00

- I want to capture the negative-validation and accessibility lane results while they are fresh, because they materially change the Loop 1 defect picture and show both fixed behavior and still-open gaps from PR #234.

the actions you take - include urls, steps and data that you use, describe any UI interactions that you make

- Reviewed the returned `negative-validation-test-log.md` and `responsive-accessibility-test-log.md`.
- Recorded the repeatable text-mode validator results for:
 - `date.between(from=1609459200000, to=1577836800000)`
 - `number.int(max=5, min=10)`
 - `location.cardinalDirection(abbreviated=maybe)`
 - `finance.iban(formatted=true, formatted=false)`
- Recorded the repeatable docs/runtime mismatch where the deployed Faker Helpers docs expose `domain.helpers.arrayElement(["red", "green", "blue"])` but the live generator rejects it.
- Recorded the repeatable accessibility behavior where help popups on `app.html` and `generator.html` visually expose links or buttons but Tab moves focus back into the main page and dismisses the popup instead of entering the popup content.
- Created defect files for the keyboard reachability issue, the stale Faker Helpers docs example, and the row-mode/text-mode params inconsistency.

the observations and results that you make

- The new ordered-bounds validation work appears healthy in generator text mode for the exercised `date.between` and `number.int` cases.
- The current deployed environment still has at least three credible defects:
 - shared help popups expose non-keyboard-reachable interactive content
 - the deployed Faker Helpers docs still show a rejected `domain.helpers.arrayElement` example
 - row-mode domain params handling can diverge from text mode and hide the real ordered-bounds failure behind a misleading missing-argument message
- Loop 1 now contains both positive regression evidence and confirmed defect evidence rather than just route checking.

2026-06-22 13:49 +01:00

- I want to run Loop 2 against the remaining gaps from Loop 1, especially nested `/site/` navigation breadth, docs-surface consistency, and any newly exposed public-content issues that were not part of the original route dispute.

the actions you take - include urls, steps and data that you use, describe any UI interactions that you make

- Reviewed Loop 1 coverage and generated these Loop 2 ideas, then classified them:
 - **execute-now** verify nested `/site/` home route via the **AnyWayData** navbar link
 - **execute-now** verify nested `/site/blog` route via the docs/blog navbar
 - **execute-now** verify nested `/site/privacy` route via the footer
 - **execute-now** verify the deployed **Method Picker UI Spec** route and inspect whether the published spec still matches the live picker shape
 - **execute-now** inspect `/site/` home page CTA routing and visible footer/nav paths after successful navigation
 - **execute-now** inspect the public blog listing for content/rendering anomalies once the route is proven live
 - **defer** recheck shared regex help `target=\"_blank\"` behavior in a browser path that can reliably surface new-tab creation
 - **defer** execute the published direct faker **abbreviated** example end to end
 - **defer** isolate the root-page console error
 - **defer** compare Storybook shared help/docs anchors
- Executed the **execute-now** ideas by navigating within the nested docs/site shell:
 - opened <https://eviltester.github.io/grid-table-editor/site/docs/test-data/method-pi>
 - clicked the top **Blog** navbar link to open <https://eviltester.github.io/grid-table-editor/sit>
 - clicked the footer **Privacy Policy** link to open <https://eviltester.github.io/grid-table-edit>
 - clicked the **AnyWayData** navbar link from `site/privacy` to open <https://eviltester.github.io/grid-table-editor/site/>
 - inspected the nested site home snapshot, including the **Use The Application** CTA and footer/nav routes
 - inspected the blog listing content after proving the route was live

the observations and results that you make

- The executed nested `/site/` routes are broadly healthy:
 - `site/blog`, `site/privacy`, and `site/` all resolved correctly
 - the nested site home exposes **Use The Application** to <https://eviltester.github.io/grid-tabl>
 - the nested navbar and footer routes remained under `/grid-table-editor/site/...`
- The published **Method Picker UI Spec** still aligns directionally with the live picker work already exercised in Loop 1: it describes a shared modal

picker, search, tabs, details, params/examples, and both generator and embedded grid surfaces.

- A new user-visible content defect appeared on **site/blog**: the first listed post preview renders frontmatter-like metadata text (**slug**, **authors**, **tags**, **date**) as visible article content. I recorded this as a new defect because it is public, repeatable, and not explained by the original route dispute.
- The deferred ideas remain useful for later loops because they target tooling-limited tab behavior, deeper docs/runtime execution, and broader shared-surface comparison.

2026-06-22 13:53 +01:00

- I want Loop 3 to challenge the remaining assumptions from Loop 2, especially whether the faker-helper docs concern is a real defect or a misread, and whether the blog-rendering problem also reproduces beyond the listing page.

the actions you take - include urls, steps and data that you use, describe any UI interactions that you make

- Reviewed Loop 2 gaps and generated these Loop 3 ideas:
 - **execute-now** open the individual blog post for **release-prep-combinatorial-grid-workflows**
 - **execute-now** open **site/docs/test-data/faker-test-data** and inspect the published direct faker examples
 - **execute-now** open the nested **Faker Helpers** docs route from the sidebar instead of relying on guessed paths
 - **execute-now** verify whether the helpers page presents **domain.helpers.arrayElement(...)** as recommended usage or as a warning/example of invalid usage
 - **execute-now** verify whether the command-coverage lane's guessed **faker/helpers** route is actually live
 - **defer** execute the published direct faker **abbreviated** example in the generator
 - **defer** compare the embedded **app.html** picker against generator for the same faker/domain commands
 - **defer** recheck **target="_blank"** help-link behavior in a different browser path
 - **defer** isolate the landing-page console error
 - **defer** extend row-mode validator parity checks to **number.int** and **finance.iban**
- Executed the **execute-now** items by:
 - opening <https://eviltester.github.io/grid-table-editor/site/blog/2026/06/12/release-prep-combinatorial-grid-workflows>
 - opening <https://eviltester.github.io/grid-table-editor/site/docs/test-data/faker-test-data>
 - clicking the sidebar **Faker Helpers** route to open <https://eviltester.github.io/grid-table-editor/docs/test-data/faker-test-data/faker/helpers>
 - inspecting the visible direct faker examples and helper guidance on both pages

the observations and results that you make

- The blog-rendering defect is stronger than first thought: the individual post page also renders frontmatter-like metadata (`slug`, `authors`, `tags`, `date`) as visible content, not just the listing preview.
 - The Faker Helpers route is live under `site/docs/test-data/faker/helpers`; the earlier guessed 404 concern was not a real product issue.
 - The earlier `domain.helpers.arrayElement(...)` defect candidate does not hold up after direct review:
 - the Faker Helpers page explicitly shows valid usage with `helpers.arrayElement(...)` and `faker.helpers.arrayElement(...)`
 - it separately shows `domain.helpers.arrayElement(...)` inside a **Do not use** warning block and explains that it is invalid
 - Because of that direct recheck, I removed the earlier false-positive defect candidate from the confirmed defect set.
 - The broader faker-docs drift risk still exists around `faker.location.cardinalDirection({ abbreviated: true })`, but it remains a follow-up risk rather than a confirmed defect until it is executed end to end against runtime behavior.
-

2026-06-22 13:57 +01:00

- I want to perform the mandatory final review loop before packaging so the stop decision is evidence-based and any lingering false positives, route gaps, or auxiliary content-page issues are resolved while the browser context is still warm.

the actions you take - include urls, steps and data that you use, describe any UI interactions that you make

- Reviewed the story, PR summary, changed-surface inventory, accumulated logs, confirmed defects, suspicious risks, sampled command families, reviewed docs, and deferred ideas.
- Generated these final-review ideas and classified them:
 - **execute-now** open `site/about`
 - **execute-now** open `site/contact`
 - **execute-now** verify the live **Faker Helpers** route from sidebar navigation
 - **execute-now** verify the **Faker Based Data** page and inspect the published direct faker examples
 - **execute-now** verify whether the individual `release-prep-combinatorial-grid-workflows` blog post repeats the frontmatter-rendering problem
 - **defer** execute `faker.location.cardinalDirection({ abbreviated: true })` end to end in runtime
 - **defer** isolate the landing-page console error
 - **defer** recheck `target=\"_blank\"` tab-creation behavior in a different browser path

- **defer** extend row-mode validator parity checks to additional commands
- **defer** inspect Storybook shared help/docs anchors
- Executed the **execute-now** items by navigating to:
 - <https://eviltester.github.io/grid-table-editor/site/about>
 - <https://eviltester.github.io/grid-table-editor/site/contact>
 - <https://eviltester.github.io/grid-table-editor/site/docs/test-data/faker-test-data/>
 - <https://eviltester.github.io/grid-table-editor/site/docs/test-data/faker/helpers>
 - <https://eviltester.github.io/grid-table-editor/site/blog/2026/06/12/release-prep-co>
- Reconciled those observations against the earlier defect set and removed the invalid `domain.helpers.arrayElement(...)` defect candidate after directly checking the current deployed page wording.

the observations and results that you make

- The auxiliary nested site routes `site/about` and `site/contact` are live and routed correctly under `/grid-table-editor/site/...`
- The nested **Faker Helpers** route is live and does not present `domain.helpers.arrayElement(...)` as recommended usage. It explicitly warns **Do not use** that form, so the earlier helpers-page defect candidate was invalid.
- The nested **Faker Based Data** page still presents direct faker examples including `faker.location.cardinalDirection({ abbreviated: true })`, but because that path was not executed end to end in runtime it remains a follow-up risk rather than a confirmed defect.
- The frontmatter-rendering problem is confirmed on both the blog listing and the individual **release-prep-combinatorial-grid-workflows** post page, making it a stronger content defect than originally thought.
- Stopping is now justified because:
 - coverage is broad across routing, docs/help, command families, negative validation, UX, and accessibility
 - multiple explicit loops were completed
 - recent loops are still refining findings, but mainly by validating or invalidating edge cases rather than opening large new coverage areas
 - the remaining gaps are mostly tooling-limited or lower-yield follow-up paths already captured as deferred ideas

command-coverage-test-log.md

Command Coverage Test Log

Charter

- Owner: subagent for command coverage and example execution
 - Goal: sample representative command families changed or risk-affected by PR #234 using the deployed test environment only
 - Write scope: this file only
 - Techniques and heuristics: exploratory testing, risk-based sampling, equivalence partitioning, boundary analysis, documentation testing, consistency checking
-

2026-06-22T13:45:26+01:00

- What you think you want to do and why

I wanted to confirm the deployed browser path, map the live generator and docs surfaces, and harvest executable command examples from the published help/docs before attempting broad positive command execution. This was the fastest way to get evidence-backed coverage across changed command families without guessing syntax.

the actions you take - include urls, steps and data that you use, describe any UI interactions that you make

Used Playwright CLI browser automation against the deployed environment only.

Opened <https://eviltester.github.io/grid-table-editor/> and captured the landing-page snapshot.

Inspected the live test-environment landing page and confirmed visible links for:

- `./app.html`
- `./generator.html`
- `./combinatorial.html`
- `./webmcp.html`
- `./writer-schema.html`
- `./storybook/index.html`
- `./site/`

Opened <https://eviltester.github.io/grid-table-editor/generator.html> and captured a fresh snapshot.

Confirmed the row-mode generator surfaces and controls visible in the deployed app:

- schema row with `regex` selected by default
- type options `enum`, `literal`, `regex`, `domain`, `faker`
- help link target for regex row: <https://eviltester.github.io/grid-table-editor/site/docs/test-c>
- `Generate Data` and `Preview` controls

Clicked `Edit as Text` in the generator and opened the schema help tooltip.

Read the text-mode help content shown in the deployed app. The live help popup included the exact example schema:

- `First Name`
- `person.firstName`
- `Status`
- `enum(active,inactive,pending)`

Reviewed the published docs using browser navigation only:

- <https://eviltester.github.io/grid-table-editor/site/docs/intro>
- <https://eviltester.github.io/grid-table-editor/site/docs/category/generating-data>
- <https://eviltester.github.io/grid-table-editor/site/docs/test-data/domain/domain-test-c>
- <https://eviltester.github.io/grid-table-editor/site/docs/test-data/faker-test-data>

Extracted representative examples from the live docs for command-family sampling:

- domain examples from the published `domain-test-data` page:
 - `person.firstName()`
 - `person.lastName()`
 - `internet.email()`
 - `location.streetAddress()`
 - `location.cardinalDirection(abbreviated=true)`
 - `date.between(from=1577836800000, to=1659312000000)`
 - `finance.iban(formatted=true, countryCode="GB")`
 - `finance.iban(formatted=false, countryCode="DE")`
 - `number.int(min=32, max=47)`
- faker examples from the published `faker-test-data` page:
 - `helpers.mustache("I found {{count}} instances.", { count: () => `${this.number.int()}` })`
 - `helpers.fake("Hi, my name is {{person.firstName}} {{person.lastName}}!")`
 - `faker.location.cardinalDirection({ abbreviated: true })`

Checked additional route behavior directly in the browser by opening likely docs URLs:

- <https://eviltester.github.io/grid-table-editor/site/docs/test-data/> returned GitHub Pages 404
- <https://eviltester.github.io/grid-table-editor/site/docs/test-data/faker-helpers> returned GitHub Pages 404

the observations and results that you make

Concrete coverage achieved:

- deployed landing page coverage for the `testenv` entry points
- generator UI coverage for row mode and text mode
- in-app help coverage for text schema examples
- published docs coverage for `generating-data`, `domain`, and `faker` command families
- representative positive example collection across `domain`, `faker/helper`, `regex`, `enum`, and literal-related surfaces

Concrete command families sampled from live browser evidence:

- domain families: `person`, `internet`, `location`, `date`, `finance`, `number`
- `faker/helper` families: `helpers.fake`, `helpers.mustache`, direct `faker` object-style example for `location.cardinalDirection`
- `regex` family: `regex` help route present from the generator row
- `enum` family: live text-mode help showed `enum(active,inactive,pending)`
- literal family: visible docs/category coverage and generator type option present

Concrete findings so far:

- The deployed generator text-mode help is live and gives a usable example schema, which is good evidence that the text-mode command examples are reachable in the current build.
- The published `domain-test-data` page is live and includes multiple parameterized examples with ordered bounds and structured arguments, which is directly relevant to the PR risk area.
- The published `faker-test-data` page is live and clearly differentiates `faker-helper` usage from domain abstractions, which is relevant to removed/deprecated boundary questions.
- A guessed base route `site/docs/test-data/` is not a valid docs entry point in the deployed environment and returns `404`; category navigation appears to be required instead.
- A guessed direct route `site/docs/test-data/faker-helpers` also returned `404`; I did not confirm whether this is a true missing page or a route-name mismatch because I stopped before deeper route discovery.

Repeatability notes:

- The docs pages listed above were repeatably reachable during this pass.
- The two guessed routes that returned `404` were repeatable during this pass.

Confidence/risk note:

- I gathered real browser evidence for coverage and examples, but broad execution of the multiline mixed schema in the generator stalled on Playwright CLI argument/quoting friction rather than an app-visible validation error. I am not treating that as a product defect.

2026-06-22T13:52:30+01:00

- What you think you want to do and why

I wanted to close the lane with a useful partial result rather than spend more time fighting the browser CLI wrapper. The goal for this final entry was to record the concrete stop point, coverage achieved, and the highest-value next command ideas for later execution.

the actions you take - include urls, steps and data that you use, describe any UI interactions that you make

Reviewed the browser evidence already gathered from these deployed pages:

- <https://eviltester.github.io/grid-table-editor/>
- <https://eviltester.github.io/grid-table-editor/generator.html>
- <https://eviltester.github.io/grid-table-editor/site/docs/intro>
- <https://eviltester.github.io/grid-table-editor/site/docs/category/generating-data>
- <https://eviltester.github.io/grid-table-editor/site/docs/test-data/domain/domain-test-d>
- <https://eviltester.github.io/grid-table-editor/site/docs/test-data/faker-test-data>

Confirmed the command/example set already harvested from the live docs and in-app help for later execution:

- `person.firstName()`
- `person.lastName()`
- `internet.email()`
- `location.cardinalDirection(abbreviated=true)`
- `date.between(from=1577836800000, to=1659312000000)`
- `finance.iban(formatted=true, countryCode="GB")`
- `finance.iban(formatted=false, countryCode="DE")`
- `number.int(min=32, max=47)`
- `text-mode help example enum(active,inactive,pending)`
- `faker example helpers.fake("Hi, my name is {{person.firstName}} {{person.lastName}}!")`
- `faker example helpers.mustache("I found {{count}} instances.", { count: () => `${this.number.int()}` })`

Stopped before further command execution once the browser wrapper friction was clearly tool-side rather than product-side.

the observations and results that you make

Final partial command-coverage outcome for this lane:

- positive command coverage is partial and evidence-backed rather than speculative
- command families definitely covered at the docs/help/browser level:
 - domain
 - faker/helper
 - regex

- enum
- literal
- structured/constrained parameter examples definitely covered at the docs/help/browser level:
 - boolean flag parameter `abbreviated=true`
 - ordered bounds `from=... to=...`
 - ordered bounds `min=... max=...`
 - mixed boolean/string parameters `formatted=true, countryCode="GB"`
- multiple-example docs/help coverage was achieved for:
 - `domain-test-data`
 - `faker-test-data`
 - text-mode schema help popup in the generator

No confirmed product defects were proven by this lane.

Most important risks left open by this partial finish:

- runtime acceptance of the harvested multiline schema was not fully exercised
- removed/deprecated command visibility remains only lightly sampled
- runtime parity between docs examples and preview output remains unproven for this session

Recommended follow-up execution order from this lane:

1. execute `location.cardinalDirection(abbreviated=true)` alone
2. execute `date.between(from=1577836800000, to=1659312000000)` alone
3. execute `finance.iban(formatted=true, countryCode="GB")` and `finance.iban(formatted=false, countryCode="DE")`
4. execute `number.int(min=32, max=47)` alone
5. execute `helpers.fake("Hi, my name is {{person.firstName}} {{person.lastName}}!")`
6. compare text-mode help syntax `person.firstName` vs docs syntax `person.firstName()`

negative-validation-test-log.md

Negative Validation Test Log

Charter

- Owner: subagent for negative validation and malformed parameter testing

- Goal: probe invalid inputs, reversed bounds, malformed parameters, and row-mode vs text-mode validation behavior using the deployed test environment only
- Write scope: this file only
- Techniques and heuristics: negative testing, boundary analysis, equivalence partitioning, state comparison, consistency checking

2026-06-22T13:46:10+01:00

- What you think you want to do and why

Probe malformed domain commands and stale helper examples in the deployed generator so I can compare text-mode validation against row-mode validation, check reversed ordered bounds, invalid validators, duplicate keywords, and confirm whether docs examples that look stale are rejected clearly by the live app.

the actions you take - include urls, steps and data that you use, describe any UI interactions that you make

Used Chrome DevTools browser tooling against <https://eviltester.github.io/grid-table-editor/generator.html> and the deployed docs pages under <https://eviltester.github.io/grid-table-editor/site/docs/test-data/>

Reviewed deployed docs/examples by opening:

- <https://eviltester.github.io/grid-table-editor/site/docs/test-data/domain/domain-test-data/>
- <https://eviltester.github.io/grid-table-editor/site/docs/test-data/domain/date/>
- <https://eviltester.github.io/grid-table-editor/site/docs/test-data/domain/finance/>
- <https://eviltester.github.io/grid-table-editor/site/docs/test-data/domain/location/>
- <https://eviltester.github.io/grid-table-editor/site/docs/test-data/domain/number/>
- <https://eviltester.github.io/grid-table-editor/site/docs/test-data/faker/helpers/>

Extracted deployed examples from the docs surface to drive negative checks, including:

- `date.between(from=1577836800000, to=1609372800000)`
- `finance.iban(countryCode="GB")`
- `finance.iban(formatted=true)`
- `location.cardinalDirection(abbreviated=true)`
- `number.int(max=10, min=1)`
- `domain.helpers.arrayElement(["red", "green", "blue"])`

Ran text-mode generator checks by opening `generator.html`, clicking **Edit as Text**, setting the schema text area content, and clicking **Generate Data** for each case:

- `Date / date.between(from=1609459200000, to=1577836800000)`
- `Num / number.int(max=5, min=10)`
- `Dir / location.cardinalDirection(abbreviated=maybe)`

- `Colors / domain.helpers.arrayElement(["red", "green", "blue"])`
- `IBAN / finance.iban(formatted=true, formatted=false)`
- `IBAN / finance.iban(countryCode="ZZZ")`

Also compared row-mode behavior on a fresh `generator.html` page:

- changed the first row type from `regex` to `domain`
- observed the row UI changed from a free-text command field to a domain-command picker plus Params e.g. (10) input
- selected `date.between`
- tried params as `(from=1609459200000, to=1577836800000)`
- tried params as `from=1609459200000, to=1577836800000`
- clicked **Generate Data** after each variant

the observations and results that you make

Text-mode validation was specific and repeatable for several malformed command cases:

- `date.between(from=1609459200000, to=1577836800000)` returned Row 1: invalid domain params - Invalid keyword arguments: argument "from" must be less than or equal to argument "to". Repeatable.
- `number.int(max=5, min=10)` returned Row 1: invalid domain params - Invalid keyword arguments: argument "min" must be less than or equal to argument "max". Repeatable.
- `location.cardinalDirection(abbreviated=maybe)` returned Row 1: invalid domain params - Invalid keyword arguments: bare values are not allowed; wrap strings in quotes. Repeatable.
- `finance.iban(formatted=true, formatted=false)` returned Row 1: invalid domain params - Invalid keyword arguments: duplicate named argument "formatted". Repeatable.

The deployed docs/helper mismatch is real enough to flag for follow-up:

- the deployed Faker Helpers docs surface still exposes `domain.helpers.arrayElement(["red", "green", "blue"])`
- the live generator rejects it with Row 1: `helpers.* is faker-only; use faker.helpers.*` Row 1: unknown domain command `"domain.helpers.arrayElement"`.
- this is repeatable and looks like a stale/unsafe example even though nearby docs text says helpers are faker-only

Row-mode vs text-mode behavior differed in a potentially confusing way for the same date case:

- text mode accepted the full command syntax and produced the specific reversed-bounds validation message
- row mode splits command selection from params entry, and `date.between` params handling was inconsistent

- when params were entered as (from=1609459200000, to=1577836800000), the params field did not retain the value and the app reported Row 1: invalid domain params - Invalid keyword arguments: argument "from" is required
- when params were entered as from=1609459200000, to=1577836800000, the field retained the text and the app reported Row 1: params should be wrapped in parentheses, e.g. (from=1609459200000, to=1577836800000).
- this suggests the row-mode params input/parser path is not behaving the same way as text mode for equivalent malformed data, and the odd clearing of the parenthesized input is worth follow-up

One finance edge case stayed suspicious rather than clearly validated:

- `finance.iban(countryCode="ZZZ")` did not show a validation message
- I also did not observe generated preview data appearing in the text preview or grid preview after pressing **Generate Data**
- because the browser session was interrupted before I could chase that further, I am treating this as incomplete coverage rather than a confirmed defect

Additional environmental note:

- the Playwright MCP path was not required because Chrome DevTools browser tooling worked
- the dedicated Playwright MCP navigation path had earlier startup issues, so I stayed on the working non-Playwright browser path as instructed

docs-consistency-test-log.md

Docs Consistency Test Log

Charter

- Owner: subagent for docs/help/content consistency
- Goal: compare published docs, in-app help, visible examples, and runtime behavior for changed command families using the deployed test environment only
- Write scope: this file only
- Techniques and heuristics: documentation testing, oracle checking, consistency checking, exploratory comparison

2026-06-22T13:47:38.2027885+01:00

- Capture a partial but concrete comparison set before stopping: verify the visible help/doc links in the deployed environment, compare the published method-picker and faker docs against the live generator picker/runtime, and separate true inconsistencies from accepted out-of-scope differences.

the actions you take - include urls, steps and data that you use, describe any UI interactions that you make

- Opened <https://eviltester.github.io/grid-table-editor/> in the deployed browser and reviewed the landing page cards plus build metadata for branch 233-test-env-consistency
- Opened <https://eviltester.github.io/grid-table-editor/generator.html> and reviewed the default row-mode schema editor, including the regex help link and the schema help affordances
- Opened <https://eviltester.github.io/grid-table-editor/app.html>, expanded the Test Data section, and opened the visible help tooltip for the section
- Fetched and reviewed published docs pages reachable from the deployed environment:
 - <https://eviltester.github.io/grid-table-editor/site/docs/test-data/test-data-generation>
 - <https://eviltester.github.io/grid-table-editor/site/docs/test-data/regex-test-data>
 - <https://eviltester.github.io/grid-table-editor/site/docs/test-data/method-picker-ui>
 - <https://eviltester.github.io/grid-table-editor/site/docs/test-data/faker-test-data>
- On the faker docs page, inspected the visible example block that includes `faker.location.cardinalDirection({ abbreviated: true })`
- Back in `generator.html`, changed the schema row source type from `regex` to `faker`
- Clicked the visible `Select faker` command control to open the live `Select schema method` dialog
- In the live picker dialog, used the `Filter methods` search box with `internet.httpMethod` and then with `cardinalDirection`
- With `cardinalDirection` still filtered, clicked the `location` category tab in the picker and reviewed the resulting method details and usage examples
- Verified the published docs navbar `App` link target by requesting <https://eviltester.github.io/grid-table-editor/site/app.html>

the observations and results that you make

- The deployed landing page and in-app help links that I checked are consistently routed to the nested testenv docs/app paths rather than an external production docs host
- The `Test Data` help tooltip in `app.html` links `Learn more to` <https://eviltester.github.io/grid-table-editor/site/docs/test-data/test-data-generation> which is consistent with the testenv link-consistency goal
- The generator row-level regex help link points to <https://eviltester.github.io/grid-table-editor/s> which is also consistent

- The published `Method Picker UI Spec` is clearly labeled as a spec, but the generator runtime also now exposes a real modal picker with search, category tabs, method details, parameter sections, and usage examples, so the generator side looks directionally aligned with the published design
- The live picker is concrete evidence that users can inspect examples in runtime for at least some command families; this is stronger than the older dropdown-only surface I first saw before opening the picker
- The strongest content inconsistency I found is in the published faker docs page: it visibly includes the direct faker example `faker.location.cardinalDirection({ abbreviated: true })`
- In the live generator picker, filtering `cardinalDirection` and choosing the `location` category surfaces the domain command `location.cardinalDirection`, with method details headed `location.cardinalDirection`, a canonical call shown as `awd.domain.location.cardinalDirection()`, and usage examples for `location.cardinalDirection`
- That means the published faker example is not aligned with the live picker's preferred/exposed command surface for the same concept; this is the clearest candidate for stale example drift in the docs/help family I covered
- I did not confirm from runtime whether the direct faker `abbreviated` parameter example is still accepted when executed, only that it is still published in the docs while the picker/runtime oracles point users toward the domain command instead
- Searching the live picker for `internet.httpMethod` from the open faker-picker path did not immediately yield a selected method before I stopped; I consider that area only partially covered, not a confirmed defect
- The published docs navbar `App` link is not broken in this environment: <https://eviltester.github.io/grid-table-editor/site/app.html> returned a live app page
- I noticed non-visible page metadata on docs pages still points canonical/OG URLs at <https://anywaydata.com/>, but I did not treat that as a finding for this lane because the issue focus is visible links/content consistency rather than metadata-only differences

2026-06-22T13:38:12.6884840+01:00

- Start the docs/help consistency session by connecting to the deployed test environment, then compare visible docs/help/examples against runtime behavior for the changed command families. I want the log to show the initial setup path and any tool constraints before deeper coverage.

the actions you take - include urls, steps and data that you use, describe any UI interactions that you make

- Confirmed the allowed write target exists: `D:\github\anywaydata-ai-testing\docs\testing\20260622`

- Looked up prior memory for `grid-table-editor` issue 233, docs/help consistency work, and earlier deployed-environment findings to focus this pass on visible links, usage examples, validators, and picker/help oracles
- Read the local skill instructions for `control-in-app-browser` and `playwright`
- Attempted to bootstrap the in-app browser runtime against the deployed test surface via `C:\Users\mr_ri\.codex\plugins\cache\openai-bundled\browser\26.616.41845\sc`
- The attempted browser connection did not reach the page because the runtime failed during setup before any navigation

the observations and results that you make

- The session is correctly constrained to append-only logging in this single file
- Prior context suggests the method picker/details panel is a high-value comparison surface for command/help drift, especially around structured usage examples and invalid parameters
- The in-app browser path was not usable in this session because setup failed with `codex/sandbox-state-meta: missing field sandboxPolicy`
- Because the failure happened before page access, I treated it as an environment/tooling problem rather than a product finding and prepared to continue with Playwright-based browser automation against the same deployed URLs

ux-regression-test-log.md

UX Regression Test Log

Charter

- Owner: subagent for UX, workflow, and regression behavior
- Goal: exercise generator, method picker, help popups, tooltip interaction, and related flows affected by PR #234 using the deployed test environment only
- Write scope: this file only
- Techniques and heuristics: exploratory workflow testing, state/flow modeling, usability heuristics, regression comparison

2026-06-22T13:45:33.1145836+01:00

- What you think you want to do and why

Continue the deployed-only UX regression pass against <https://eviltester.github.io/grid-table-editor/> and focus on the generator workflow, faker/domain method picker, params editor, preview/help affordances, and the PR #234 expectation that only one tippy/help popup should stay open at a time. Capture enough evidence to either finish the lane or return a reliable partial result if browser tooling blocks deeper exploration.

the actions you take - include urls, steps and data that you use, describe any UI interactions that you make

Opened <https://eviltester.github.io/grid-table-editor/> in a real browser session and confirmed the landing page build metadata showed branch `233-test-env-consistency`, commit `ec1b8db43eb9`, built `2026-06-22T12:16:58.547Z`. Followed the `Open generator.html` link into <https://eviltester.github.io/grid-table-editor/generator.html>. On the generator page, changed the first schema row type from the default `regex` row to `faker`, then opened the method picker from `Select faker command`. Verified the picker rendered category chips, a filter box, a method list, and a details pane with schema example, parameter tables, usage examples, and docs link for `helpers.arrayElement`. Re-opened the picker and switched the category to `internet`, then selected `internet.httpMethod` and applied it so the row changed to `domain` with `internet.httpMethod` selected. Filled the column name with `method` to keep the row valid enough for downstream interactions. Opened the params editor path for `internet.httpMethod` and inspected the dialog content through the live page DOM when the accessibility snapshot truncated it; confirmed it showed `Edit Params`, the full command `awd.domain.internet.httpMethod`, parameter controls for `commonOnly` and `excludes`, and generated params text (`commonOnly=false`). For tooltip/help behavior, clicked multiple help triggers in sequence while checking visible tippy content in the DOM: `Data Generator Instructions` help, schema section help, and other help affordances around `generator/options/preview`. After each click, checked visible `[data-tippy-root]` elements and their text content to see whether a new help popup replaced the old one or stacked with it. Also attempted to continue via non-Playwright browser tooling after the user asked to reserve Playwright for fallback: Chrome DevTools reported the profile browser was already running and Playwright MCP failed to attach cleanly, so deeper follow-through stayed on the already-collected deployed evidence instead of waiting on tool recovery.

the observations and results that you make

The deployed environment is live and the generator workflow surfaces affected by PR #234 are present and usable. The method picker is a strong regression oracle here: it rendered smoothly, switching from `faker` helpers to `domain/internet` commands without obvious UI breakage, and the details pane showed command descriptions, schema examples, parameter metadata, usage examples, and documentation links as expected. The `internet.httpMethod` path specifically looks healthy at selection time: applying the picker changed the row into the

expected domain command state, enabled the params editor button, and the params editor dialog opened with the expected command-specific configuration content. The clearest regression result is the tippy/help behavior: sequential help clicks produced exactly one visible tippy popup at a time in the DOM, with the later click replacing the earlier popup content rather than leaving multiple visible tippies stacked. Concretely, **Data Generator Instructions** help showed one visible popup with overview text, then schema help showed one visible popup with schema/edit-text guidance instead of two simultaneous popups. That supports the "only one tippy open" expectation for the exercised generator help flows. I did not finish a full preview-generation assertion after applying `internet.httpMethod`, and I did not complete deeper coverage of preview help, file-generation help, or docs-link navigation because the alternate browser attachment paths were blocked and I chose to preserve time by returning a grounded partial rather than spending the lane on tool recovery. One additional note from initial page open: the test environment landing page reported one console error on load, but this session did not isolate or characterize that error.

responsive-accessibility-test-log.md

Responsive Accessibility Test Log

Charter

- Owner: subagent for responsive, mobile, and accessibility review
- Goal: assess focus order, keyboard reachability, popup accessibility, and narrow-width behavior across changed surfaces using the deployed test environment only
- Write scope: this file only
- Techniques and heuristics: accessibility heuristics, responsive heuristics, keyboard-only exploration, exploratory testing

2026-06-22T13:46:15.7764222+01:00

- Assess deployed-only keyboard reachability, help-popup accessibility, focus order, and narrow-width behavior across `app.html`, `generator.html`, `writer-schema.html`, and nested `site/`, with extra attention on whether help-popup links can actually be reached from the keyboard.

the actions you take - include urls, steps and data that you use, describe any UI interactions that you make

Used the deployed environment only at <https://eviltester.github.io/grid-table-editor/> and moved between `app.html`, `generator.html`, `writer-schema.html`, and `site/`. On `app.html`, I tabbed from page load, opened the top Instructions help popup, then tabbed again to see whether focus entered the popup. I also opened the Import/Export format help popup and checked whether its `Data Formats docs` link could be reached before the popup dismissed. On `generator.html`, I opened the file-generation help popup and checked whether its `Generate To File docs` link was keyboard reachable after opening the popup. I also checked for skip links on `generator.html`, `writer-schema.html`, and `site/`. For responsive coverage, I resized to 375x812 and checked `window.innerWidth`, `document.documentElement.scrollWidth`, and `document.documentElement.clientWidth` on `app.html`, `generator.html`, `writer-schema.html`, and `site/` to see whether the surfaces introduced horizontal overflow. On `site/` at narrow width, I also checked whether the mobile navigation toggle appeared. I attempted to switch to non-Playwright browser paths when asked, but the in-app browser route failed during setup and the Chrome DevTools route was blocked by an already-running browser profile, so I finished the pass with the already-working fallback rather than stop coverage.

the observations and results that you make

`app.html`: the page exposed multiple help buttons, but the help popup interaction is not keyboard-friendly. After opening the Instructions help popup, pressing `Tab` moved focus to `Add Row` in the main toolbar instead of into the popup's own `Load Sample Data` button. After opening the Import/Export format help popup, pressing `Tab` moved focus to the main-page `Markdown` format link and the popup dismissed, so the popup's `Data Formats docs` link did not become keyboard reachable in the tested flow. This is a concrete accessibility risk and matches the issue comment concern about help-popup links. At 375x812, the app did not report horizontal overflow by document-width metrics, but the page still presents a very dense sequence of controls before the grid and preview content.

`generator.html`: this surface had a working `Skip to main content` link and exposed the shared schema/editor controls. Opening `Show file generation help` revealed a popup containing a `Generate To File docs` link, but pressing `Tab` moved focus to `Generate Data` and the popup closed, so that popup link was also not keyboard reachable in the tested path. At 375x812, the page did not report horizontal overflow. The generator therefore looks more resilient than the app on mobile width, but it appears to share the same help-popup keyboard-reachability problem.

`writer-schema.html`: at 375x812, the page did not report horizontal overflow, and the shared schema editor rendered. I did not find a skip link on this page, unlike `generator.html` and `site/`. The page also showed `Writer API is not available in this browser session`, so the actual browser-Writer generation path could not be exercised in this environment; coverage here is therefore limited to the surrounding page structure, responsive width, and presence of the shared

schema surface rather than the AI generation interaction itself.

site/: the nested site exposed a **Skip to main content** link and, at 375x812, switched to a mobile nav pattern with a **Toggle navigation bar** control. Width checks did not show horizontal overflow. This surface looked materially better prepared for narrow-width use than the app/editor pages. I did not pursue third-party iframe accessibility beyond noting that a YouTube player is embedded on the home page and can become part of the focusable surface.

Cross-surface result: the strongest issue I found is that help popups can expose links or buttons visually while not moving keyboard focus into the popup after it opens. I reproduced that pattern on both **app.html** and **generator.html**, including popup content that visibly contains documentation links. That makes the current help affordance partially mouse-dependent even when the popup content itself contains focusable elements.



routing-consistency-test-log.md

Routing Consistency Test Log

Charter

- Owner: subagent for cross-surface routing consistency and verify-after-fix route checks
- Goal: verify visible help/docs links and related destinations across root pages, nested /site pages, and shared editor surfaces using the deployed test environment only
- Write scope: this file only
- Techniques and heuristics: consistency checking, exploratory testing, route-oracle checking, repeatability checks



2026-06-22T13:47:26+01:00

- What you think you want to do and why

I want to do a partial cross-surface routing pass on the deployed test environment and capture only concrete browser-verified evidence for visible help/docs/app links, because the review focus is issue 233 link consistency across root pages, nested /site pages, and shared schema/help surfaces.

the actions you take - include urls, steps and data that you use, describe any UI interactions that you make

Used the deployed environment only at <https://eviltester.github.io/grid-table-editor/> with Playwright browser tooling. Opened the root page and took a fresh snapshot of visible links. On the root page I recorded the visible destinations for `site/`, `Open app.html`, `Open generator.html`, `Open combinatorial.html`, `Open webmcp.html`, `Open writer-schema.html`, `Open Storybook`, and `Open site/`. I then clicked the visible `Open app.html` link from the root page and verified the resulting browser URL became <https://eviltester.github.io/grid-table-editor/app.html>.

On <https://eviltester.github.io/grid-table-editor/app.html> I opened the `Instructions` help toggle and enumerated visible anchors from the live page. This exposed external library/source links plus one internal shared-schema help destination with `aria-label="Regex data help"` pointing to <https://eviltester.github.io/grid-table-editor/site/docs/test-data/regex-test-data>. I also recorded that the export-format tabs (`Markdown`, `CSV`, `Delimited`, `JSON`, `JSONL`, `XML`, `SQL`, `Code`, `Code (Unit Test)`, `Gherkin`, `HTML`, `ASCII`) are in-page `#` links on `app.html`, which look like format switchers rather than docs routes.

On <https://eviltester.github.io/grid-table-editor/generator.html> I enumerated visible anchors, then took a fresh snapshot. The visible links were `Skip to main content` and the shared-schema help icon `Regex data help`, with the same destination <https://eviltester.github.io/grid-table-editor/site/docs/test-data/regex-test-data>. I clicked the visible `Regex data help` control from the generator page, then checked the current browser URL and open tabs. After that click, the current page remained <https://eviltester.github.io/grid-table-editor/generator.html> and the browser still showed a single open tab for the generator page, so this click did not yield a directly observable new-tab destination in the tooling session.

On <https://eviltester.github.io/grid-table-editor/writer-schema.html> I enumerated visible anchors from the live page. The visible internal help destination was again the shared-schema help icon `Regex data help` targeting <https://eviltester.github.io/grid-table-editor/site/docs/test-data/regex-test-data>.

On <https://eviltester.github.io/grid-table-editor/site/> I enumerated the visible navigation and CTA destinations from the live page. The visible internal routes resolved to <https://eviltester.github.io/grid-table-editor/site/>, <https://eviltester.github.io/grid-table-editor/site/app.html>, <https://eviltester.github.io/grid-table-editor/site/docs/intro>, <https://eviltester.github.io/grid-table-editor/site/blog>, <https://eviltester.github.io/grid-table-editor/site/about>, and <https://eviltester.github.io/grid-table-editor/site/contact>. The visible external GitHub and Twitter links also resolved as expected in the extracted live hrefs. I took a snapshot confirming those links were rendered in the navbar, hero CTA, and footer, but the session was interrupted before I completed click-through verification for the `/site` navbar and footer routes.

the observations and results that you make

The deployed root page currently presents consistent visible absolute destinations for the main test-environment entry links, and the clicked `Open app.html` route resolved to the expected root-level URL `https://eviltester.github.io/grid-table-editor/app.html`.

Across the three shared-schema surfaces I covered (`app.html`, `generator.html`, and `writer-schema.html`), the visible internal help destination for regex data was consistent: all surfaced the same docs target `https://eviltester.github.io/grid-table-editor/site/docs/test-data/regex-test-data`. That is good evidence that the shared help/docs routing seam is aligned across those covered pages.

The `app.html` format tabs currently look like intentional in-page controls rather than cross-page docs links because they all resolve to `app.html#`. I did not treat those as defects in this partial pass.

The generator-page click on the visible `Regex data help` icon did not create an observable new tab or change the current browser URL in the Playwright tooling session, even though the live href on that control points to the expected `/site/docs/test-data/regex-test-data` destination and the same href appears consistently on other shared-schema pages. Because that behavior could be a tooling/session limitation around `target=\"_blank\"` handling rather than a deployed routing defect, I do not consider it a confirmed issue from this evidence alone.

The nested `/site` surface appears internally consistent in the extracted live hrefs: navbar, CTA, and footer links all resolve under `/grid-table-editor/site/...` rather than leaking to the root app path, except for the intentional `/site/app.html` app entry. I did not complete click-through confirmation for those `/site` links before the interruption, so those routes remain partially covered rather than fully proven by navigation.



defects\defect-001-help-popup-keyboard-reachability.md

Defect 001: Help Popup Links And Buttons Are Not Keyboard Reachable

Summary

Shared help popups on the deployed `app.html` and `generator.html` surfaces can display focusable content such as docs links or action buttons, but keyboard focus does not move into the popup after opening it. Pressing `Tab` moves focus back into the main page flow and the popup dismisses, making the popup content effectively mouse-dependent.

Surfaces

- <https://eviltester.github.io/grid-table-editor/app.html>
- <https://eviltester.github.io/grid-table-editor/generator.html>
- likely shared help-popup behavior across related flows because both surfaces reproduce the issue

Why This Matters

- Issue comments explicitly called out keyboard reachability of help-popup links as needing investigation.
- The current behavior prevents keyboard-only users from reaching visible help links and buttons.
- This is an accessibility problem on shared help affordances, not just a cosmetic UX issue.

Repeatability

- Repeatable on the deployed environment in this session.
- Reproduced in both `app.html` and `generator.html`.

Reproduction

Case A: `app.html` top Instructions popup

1. Open <https://eviltester.github.io/grid-table-editor/app.html>.
2. Use keyboard navigation to focus the top Instructions help button.
3. Activate the help button to open the popup.
4. Press Tab.

Case B: `app.html` Import/Export format help popup

1. Open <https://eviltester.github.io/grid-table-editor/app.html>.
2. Focus and activate the Import/Export format help popup.
3. Observe that the popup visually contains the Data Formats docs link.
4. Press Tab.

Case C: `generator.html` file-generation help popup

1. Open <https://eviltester.github.io/grid-table-editor/generator.html>.
2. Focus and activate Show file generation help.
3. Observe that the popup visually contains the Generate To File docs link.
4. Press Tab.

Observed

- Case A: focus moved to **Add Row** in the main toolbar instead of into the popup's own interactive content.
- Case B: focus moved to the main-page **Markdown** format link and the popup dismissed, so the popup's **Data Formats docs** link was not keyboard reachable.
- Case C: focus moved to **Generate Data** and the popup closed, so the popup's **Generate To File docs** link was not keyboard reachable.

Expected

- After opening a help popup that contains focusable content, keyboard focus should move into the popup or at minimum allow tabbing into the popup content before dismissal.
- Visible popup links and buttons should be reachable and operable using keyboard-only interaction.

Evidence

- Responsive/accessibility subagent log: ../responsive-accessibility-test-log.md
- Main log summary: ../issue-233-test-log.md

Notes For Investigation

- This likely lives in the shared help/tippy interaction layer rather than a single page implementation.
- The PR already changed tooltip behavior to keep only one tippy open; keyboard focus management may need a companion fix in the same shared seam.

defects\defect-003-row-mode-domain-params-inconsistent-validation.md

Defect 003: Row-Mode Domain Params Behave Inconsistently Compared With Text Mode

Summary

Equivalent malformed `date.between` input produces clear ordered-bounds validation in text mode, but row mode behaves inconsistently. Parenthesized params

are not retained and produce a misleading missing-argument message, while non-parenthesized params are retained and produce the expected wrapping-format message.

Surface

- <https://eviltester.github.io/grid-table-editor/generator.html>

Why This Matters

- PR #234 explicitly added ordered-bounds validation across multiple command families.
- Text mode demonstrates that the validator is working.
- Row mode appears to have a separate params-entry/parsing issue that can hide the real validation result and confuse users.

Repeatability

- Repeatable in the negative-validation lane for the tested `date.between` case.

Reproduction

Baseline Text-Mode Behavior

1. Open <https://eviltester.github.io/grid-table-editor/generator.html>.
2. Switch to text mode.
3. Enter a schema row using `date.between(from=1609459200000, to=1577836800000)`.
4. Generate data.

Observed baseline:

- Row 1: invalid domain params - Invalid keyword arguments: argument "from" must be less than or equal to argument "to"

Row-Mode Behavior

1. Open a fresh <https://eviltester.github.io/grid-table-editor/generator.html>.
2. Leave row mode active.
3. Change the first row type from `regex` to `domain`.
4. Select `date.between`.
5. In the params field, enter `(from=1609459200000, to=1577836800000)`.
6. Generate data.
7. Repeat with `from=1609459200000, to=1577836800000` without parentheses.

Observed

- With parenthesized params:
 - the params field did not retain the typed value
 - the app reported **argument "from" is required**
- With non-parenthesized params:
 - the params field retained the typed value
 - the app reported **params should be wrapped in parentheses, e.g. (from=1609459200000, to=1577836800000).**

Expected

- Row mode should preserve the entered params consistently and either:
 - pass them through to the ordered-bounds validator, producing the same reversed-bounds message as text mode, or
 - at minimum fail in a way that does not erase validly formatted input and replace it with a misleading missing-argument error

Evidence

- Negative validation subagent log: ../negative-validation-test-log.md

Notes For Investigation

- This may be a row-mode params parsing or state-retention issue rather than a validator bug.
- It is worth retesting on other validator-backed commands such as `number.int` and `finance.iban` to see whether the inconsistency is generic to row-mode params handling.

defects\defect-004-blog-listing-shows-frontmatter.md

Defect 004: Site Blog Listing Renders Frontmatter As Visible Article Content

Summary

The nested site blog listing currently renders frontmatter-like metadata as visible article content for the first listed post instead of presenting clean blog copy.

Surface

- <https://eviltester.github.io/grid-table-editor/site/blog>

Why This Matters

- This is a user-visible content defect on the deployed nested site.
- It reduces trust in the merged `/site/` publishing surface that PR #234 touches through shared `site/testenv` build behavior.
- It suggests at least one blog entry is being rendered from malformed or unprocessed content.

Repeatability

- Observed directly and repeatably in this session on the current deployed blog listing page.

Reproduction

1. Open <https://eviltester.github.io/grid-table-editor/site/blog>.
2. Inspect the first listed post on the page.

Observed

The first visible entry shows a heading-like block containing frontmatter-style text:

- `slug: combinatorial-grid-workflows authors: [alan] tags: [release, feature, combinatorial, schema, import, export, ux] date: 2026-06-12T10:00`

This appears in the public article preview instead of normal post content.

Expected

- Blog listing previews should render article title, summary, author/date, and normal body excerpt content.
- Frontmatter metadata should not be exposed as visible article content.

Evidence

- Loop 2 live browser observation on `site/blog`
- Main log: `../issue-233-test-log.md`

Notes For Investigation

- Compare this post with other blog entries on the same page, which appear to render more normally.
- Check whether the issue is specific to a single post file or a broader markdown/frontmatter rendering problem in the nested site pipeline.